

CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer
Science & Engineering

CSCE 633: Machine Learning

Reinforcement Learning

Texas A&M University

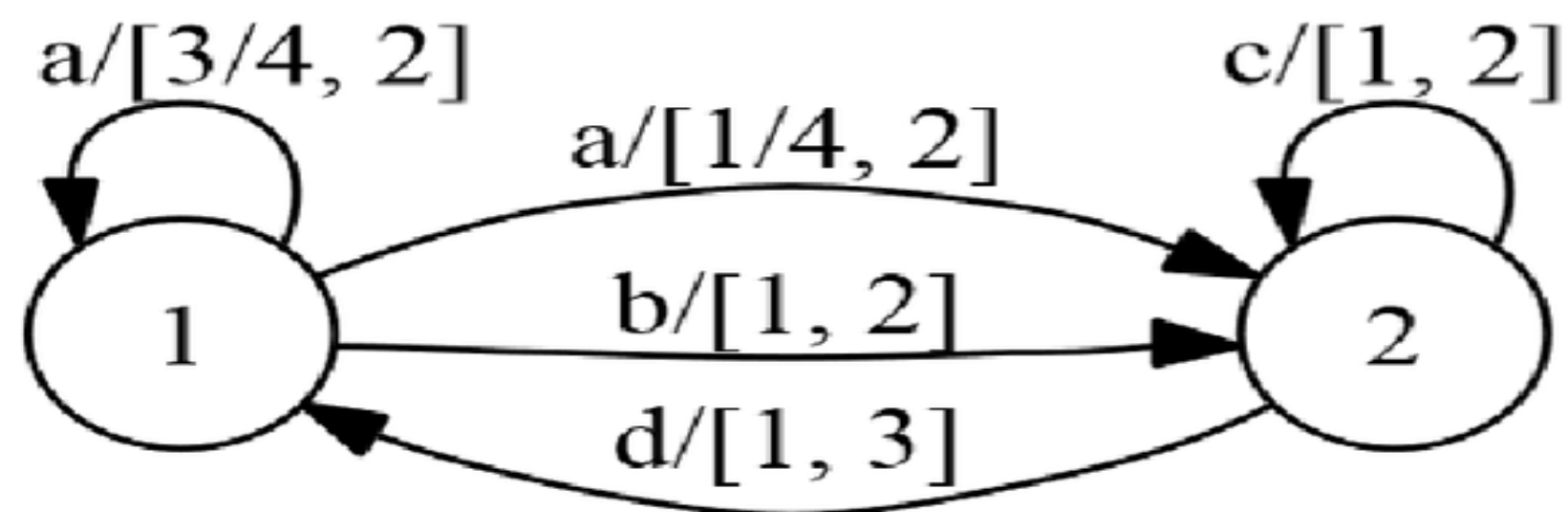
Topics

- What is reinforcement learning?
- An example application and Markov Decision Process
- Value Iteration
- Q Learning (and Policy Iteration)

Value Iteration

- Find optimal policy at each state, therefore, optimal across all states.
- Based upon Bellman equations.
- $V \leftarrow V_0$
- while $\|V - \phi(V)\| \geq \frac{(1-\gamma)\epsilon}{\gamma}$ do
- $V \leftarrow \phi(V)$
- return $\phi(V)$
- Start with an arbitrary policy, then iteratively improve via some function.

VI Algorithm - Example



$$\mathbf{V}_{n+1}(1) = \max \left\{ 2 + \gamma \left(\frac{3}{4} \mathbf{V}_n(1) + \frac{1}{4} \mathbf{V}_n(2) \right), 2 + \gamma \mathbf{V}_n(2) \right\}$$

$$\mathbf{V}_{n+1}(2) = \max \left\{ 3 + \gamma \mathbf{V}_n(1), 2 + \gamma \mathbf{V}_n(2) \right\}.$$

For $\mathbf{V}_0(1) = -1, \mathbf{V}_0(2) = 1, \gamma = 1/2, \mathbf{V}_1(1) = \mathbf{V}_1(2) = 5/2$.

But, $\mathbf{V}^*(1) = 14/3, \mathbf{V}^*(2) = 16/3$.

Value iteration (1)

Value iteration

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat

$\Delta \leftarrow 0$

 For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

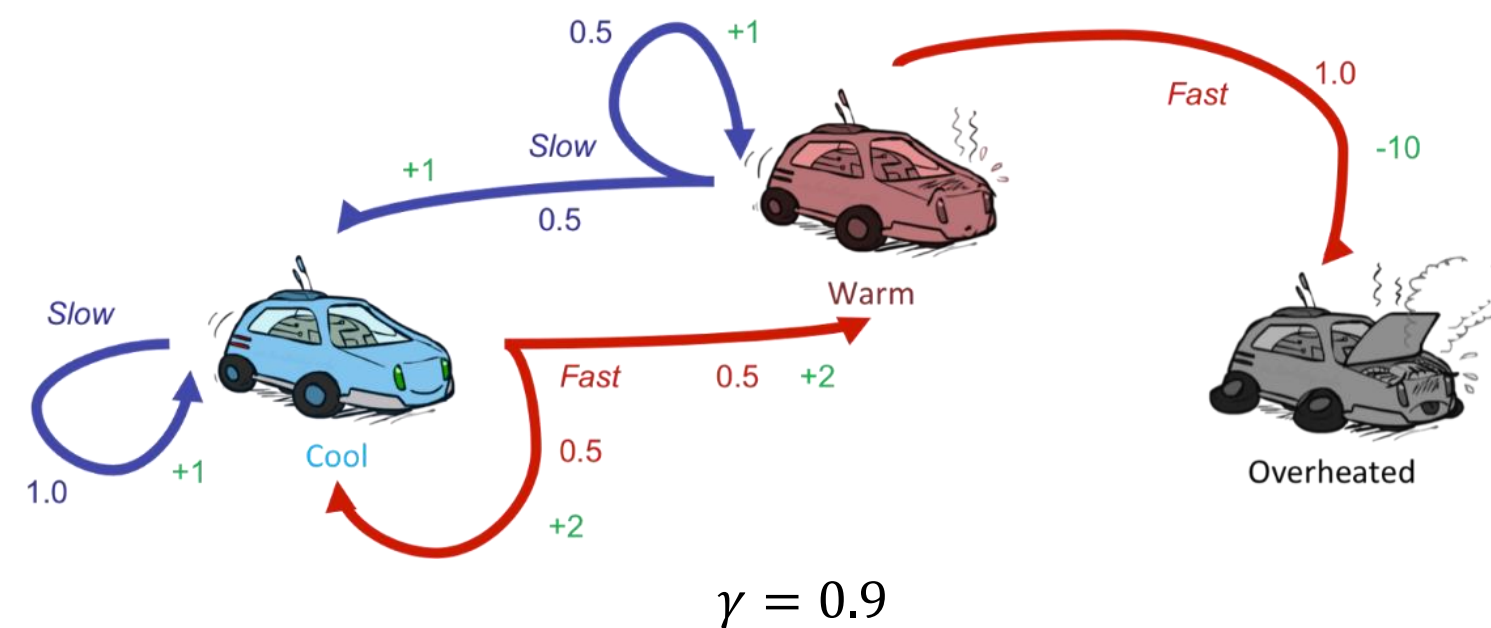
Output a deterministic policy, $\pi \approx \pi_*$, such that

$\pi(s) = \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

Courtesy: Dr. Sharon

Value Iteration (2)

			
V_0	0	0	0
V_1	2	1	0
V_2	3.5	2.5	0



Bellman operator

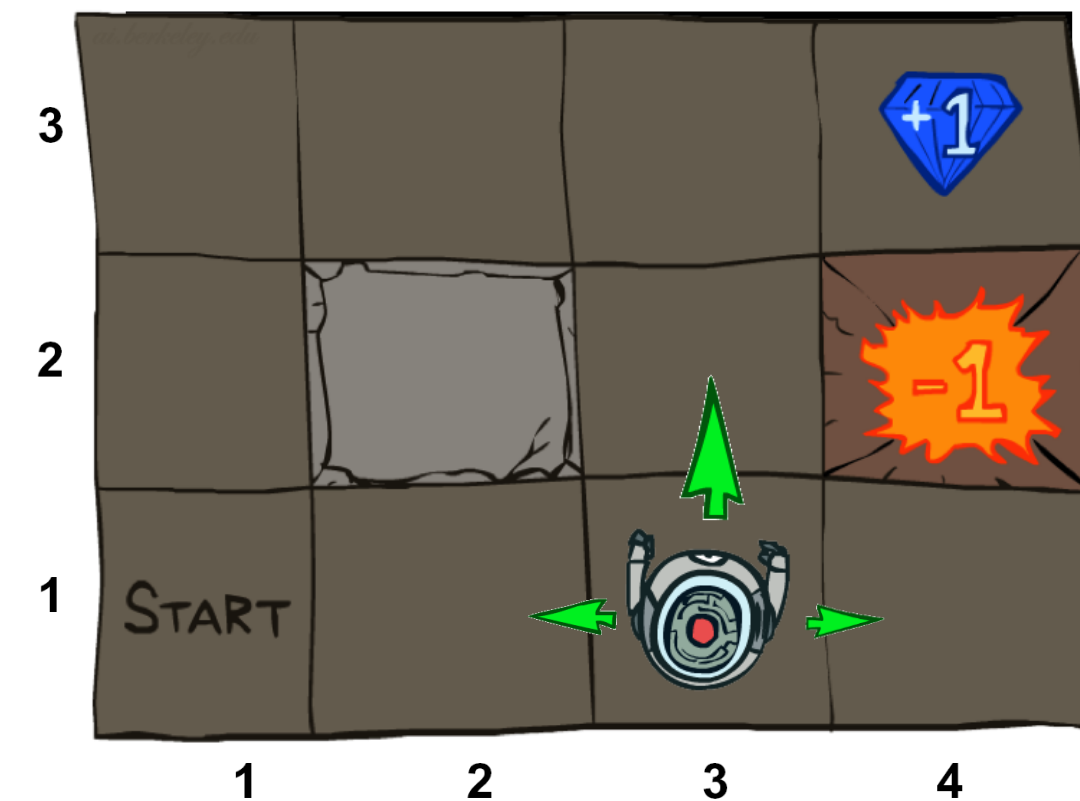
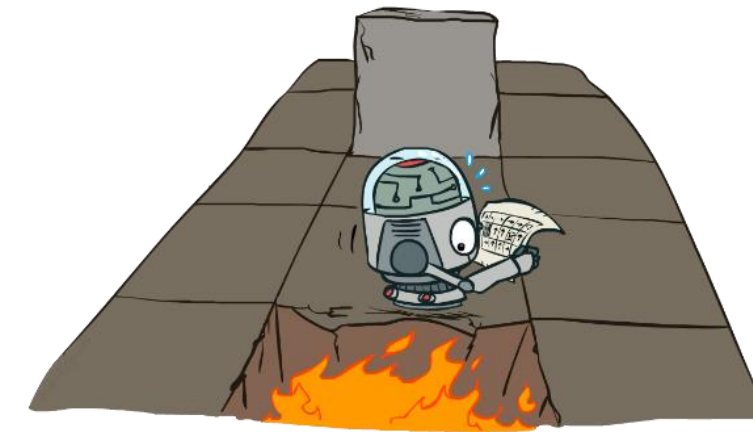
$$V_{k+1}(s) \leftarrow \max_a \left[r(s, a) + \sum_{s'} p(s'|s, a) \gamma V_k(s') \right]$$

$$= \max_a \left[\sum_{s'} p(s'|s, a) (r(s, a, s') + \gamma V_k(s')) \right]$$

Courtesy: Dr. Sharon

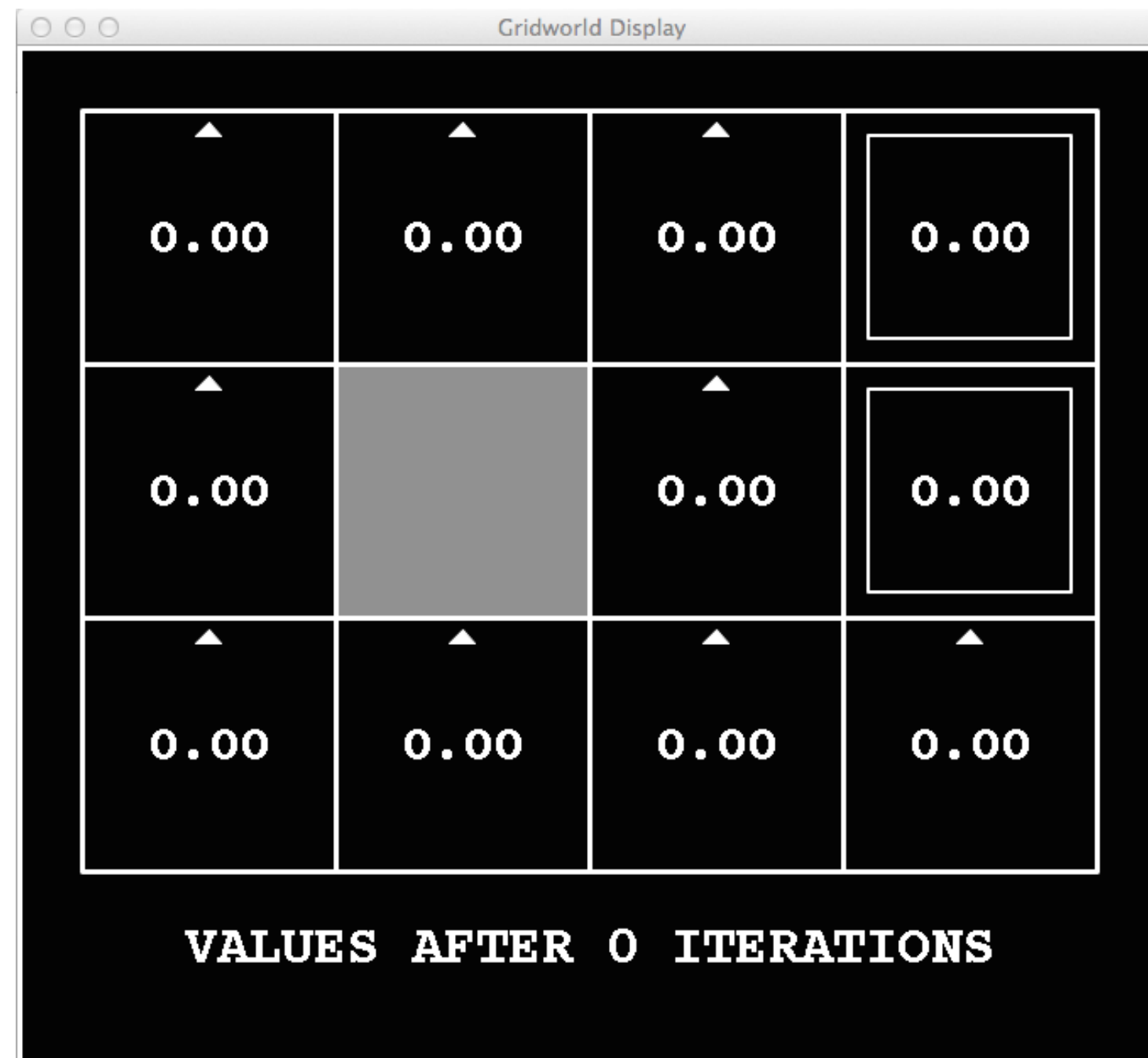
Example: Grid World

- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Noisy movement: actions do not always go as planned
 - 80% of the time, the action North takes the agent North
 - 10% of the time, North takes the agent West; 10% East
 - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
 - Small negative reward each step (battery drain)
 - Big rewards come at the end (good or bad)
- Goal: maximize sum of (discounted) rewards



Courtesy: Dr. Sharon

k=0



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

$k=1$



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

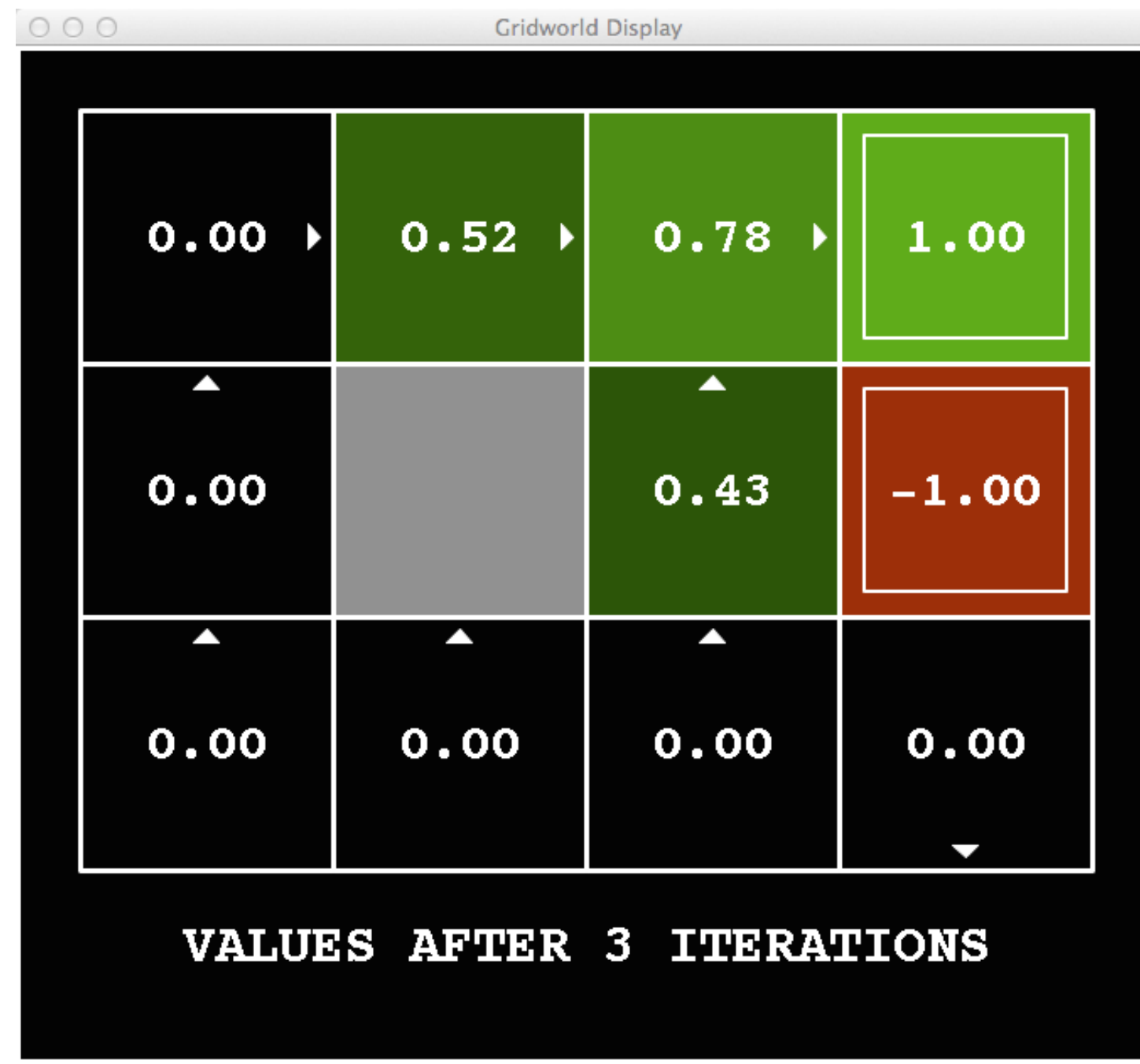
$k=2$



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=3



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

$k=4$



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=5



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=6



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

$k=7$



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=8



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=11



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=12



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

k=100



Noise = 0.2
Discount = 0.9
Living reward = 0

Courtesy: Dr. Sharon

Problems with Value Iteration

- Value iteration repeats the Bellman updates:
- $V_{k+1}(s) \leftarrow \max_a [R(s, a) + \sum_{s'} P(s'|s, a) \gamma V_k(s')]$
$$= \max_a \left[\sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s')) \right]$$
- **Issue 1:** It's slow – $O(S^2A)$ per iteration
 - Do we really need to update every state at every iteration?
- **Issue 2:** A policy cannot be easily extracted
 - Policy extraction?
- **Issue 3:** The policy often converges long before the values
 - Can we identify when the policy converged?
- **Issue 4:** requires knowing the model, $P(s'|s, a)$, and the reward function, $R(s, a)$
- **Issue 5:** requires discrete (finite) set of actions
- **Issue 6:** infeasible for large (or continuous) state spaces

Courtesy: Dr. Sharon

Solutions (briefly, more later...)

- **Issue 1:** It's slow – $O(S^2A)$ per iteration
 - Asynchronous value iteration
- **Issue 2:** A policy cannot be easily extracted
 - Learn q (action) values
- **Issue 3:** The policy often converges long before the values
 - Policy-based methods
- **Issue 4:** requires knowing the model and the reward function
 - Reinforcement learning
- **Issue 5:** requires discrete (finite) set of actions
 - Policy gradient methods
- **Issue 6:** infeasible for large (or continues) state spaces
 - Value function approximators

Courtesy: Dr. Sharon